

Week 1 Module – C & C++ Programming

Topics to be Covered

Students are required to Submit handwritten notes and practice for the following topics in

C and C++ Programming Language:

1. **Syntax**
2. **Conditions and Loops**
3. **Programming Problems**
 - Solve the given **Star Pattern Questions**

1. Square

```
*****
*****
*****
*****
*****
```

2. Hollow Square

```
*****
*   *
*   *
*   *
*   *
*****
```

3. Triangle

```
*
**
***
****
*****
```

4. Inverted Triangle

```
*****
****
***
**
*
```

5. Pyramid

```
  *
 ***
*****
*****
*****
```

6. Inverted Pyramid

```
*****
*****
*****
***
*
```

7. Half Diamond

```
*
**
***
****
****
****
***
**
*
```

8. Diamond

```
  *
 ***
*****
*****
*****
*****
***
 ***
  *
```

C and C++ Programming

C Programming

① C Syntax and Basic structure

C is a procedural and structured programming language. Every standard C program has a precise, structured layout.

Header files (#include <stdio.h>)

This pre-processor directive includes the standard input/output library. It allows the program to use the core functions, such as `printf()` for output and `scanf()` for input.

Main function (int main())

This is the most mandatory entry point of any C program. Execution starts at the opening brace `{` of `main()`. Execution ends at the terminating semicolon of `return 0;`.

Output function (printf)

Unlike C++, C uses `printf()` function to display text on the screen. Double quotes let you pass format specifiers or escape sequences like `\n` (newline).

Semicolon (;): Every statement in C must end with a semicolon. It is a statement terminator for the compiler.

② Conditions and loops

Conditional statements (if-else): Used to make decisions. The if block runs when the condition is true. If it is false, the program skips it or executes the optional else block.

Loops (for loop): To repeat a block of code a certain number of times. It is composed of initialization, condition checking and increment/decrement expressions.

Nested loops: ~~for~~ for star patterns to print two dimensional and patterns. we need a loop in another loop.

Outer loop: Manage the rows (vertical progression)

Inner loop: Handles the columns (prints spaces or stars horizontally for the current row).

③ Programming problems

Solve the given star pattern Questions

① square

o/p:—

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 5; // total rows and columns
```

```
    // outer loop for rows
```

```
    for (int i = 1; i <= n; i++) {
```

```
        // inner loop for columns
```

```
        for (int j = 1; j <= n; j++) {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n"); // move to the next line
```

```
    }
```

```
    return 0;
```

```
}
```

```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5
6      for (int i = 1; i <= n; i++) {
7          for (int j = 1; j <= n; j++) {
8              printf("* ");
9          }
10         printf("\n");
11     }
12
13     return 0;
14 }
```

Output

Clear

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

```
=== Code Execution Successful ===
```

② Hollow square pattern

```
#include <stdio.h>
int main () {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n) {
                printf("*");
            }
            else {
                printf(" "); // empty space
            }
        }
        printf("\n"); // next line
    }
    return 0;
}
```

O/p:-

```
* * * * *
*           *
*           *
*           *
*           *
* * * * *
```

```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5
6      for (int i = 1; i <= n; i++) {
7          for (int j = 1; j <= n; j++) {
8
9              // Print star only at the boundaries
10             if (i == 1 || i == n || j == 1 || j == n) {
11                 printf("*");
12             } else {
13                 printf(" "); // Print empty space inside
14             }
15         }
16
17         printf("\n");
18     }
19
20     return 0;
21 }
```


Output

[Clear](#)

* *

* *

* *

=== Code Execution Successful ===

③ Triangle

```
#include <stdio.h>
```

```
int main () {
```

```
    int n = 5;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        // inner loop: print stars in increasing order
```

```
        for (int j = 1; j <= i; j++) {
```

```
            printf(" * ");
```

```
        }
```

```
        // move to the next line
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

O/p:-

*

* *

* * *

* * * *

* * * * *



```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5
6      for (int i = 1; i <= n; i++) {
7
8          // Inner loop runs up to the current row number 'i'
9          for (int j = 1; j <= i; j++) {
10             printf("*");
11         }
12
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Output

Clear

*

**

=== Code Execution Successful ===

④ Inverted Triangle

```
#include <stdio.h>
int main () {
    int n = 5;
    // outer loop counts down from 'n' to 1
    for (int i = n; i >= 1; i--) {
        // inner loop: print 'i' stars
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

O/p:-

```
* * * * *
 * * * *
  * * *
   * *
    *
```



```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5
6      // Outer loop counts down from 'n' to 1
7      for (int i = n; i >= 1; i--) {
8          for (int j = 1; j <= i; j++) {
9              printf("*");
10             }
11
12             printf("\n");
13         }
14
15         return 0;
16     }
```

Output

Clear

**

*

=== Code Execution Successful ===

5

Pyramid

```
#include <stdio.h>
int main () {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        // print leading spaces
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        // print stars based on odd numbers formula
        for (int k = 1; k <= (2 * i - 1); k++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

O/p:-

```
      *
    * * *
  * * * *
* * * * *
* * * * *
```

```
2
3 int main() {
4     int n = 5;
5
6     for (int i = 1; i <= n; i++) {
7
8         // Print leading spaces
9         for (int j = 1; j <= n - i; j++) {
10             printf(" ");
11         }
12
13         // Print stars
14         for (int k = 1; k <= (2 * i - 1); k++) {
15             printf("*");
16         }
17
18         printf("\n");
19     }
20
21     return 0;
22 }
```


Output

Clear

*

=== Code Execution Successful ===

⑥ Inverted pyramid

```
#include <stdio.h>
int main () {
    int n = 5;
    // outer loop handles rows in reverse order
    for (int i = n; i >= 1; i--) {
        // print leading spaces
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        // print stars
        for (int k = 1; k <= (2 * i - 1); k++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

O/p:-

```

* * * * *
 * * * *
  * * *
   * *
    *
```



```
3 int main() {
4     int n = 5;
5
6     // Outer loop
7     for (int i = n; i >= 1; i--) {
8
9         // Print spaces
10        for (int j = 1; j <= n - i; j++) {
11            printf(" ");
12        }
13
14        // Print stars
15        for (int k = 1; k <= (2 * i - 1); k++) {
16            printf("*");
17        }
18
19        printf("\n");
20    }
21
22    return 0;
23 }
```

Output

Clear

*

=== Code Execution Successful ===

⑦ Half diamond

```
#include <stdio.h>
int main () {
    int n = 5;
    // upper half : Creating triangle
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    // lower half : Shrinking inverted triangle
    for (int i = n-1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

O/p:-

```
*
**
***
****
*****
*****
****
***
**
*
```

```
3 int main() {
4     int n = 5;
5
6     // Upper half
7     for (int i = 1; i <= n; i++) {
8         for (int j = 1; j <= i; j++) {
9             printf("*");
10        }
11        printf("\n");
12    }
13
14    // Lower half
15    for (int i = n - 1; i >= 1; i--) {
16        for (int j = 1; j <= i; j++) {
17            printf("*");
18        }
19        printf("\n");
20    }
21
22    return 0;
23 }
```


Output

Clear

*
**

**
*

=== Code Execution Successful ===

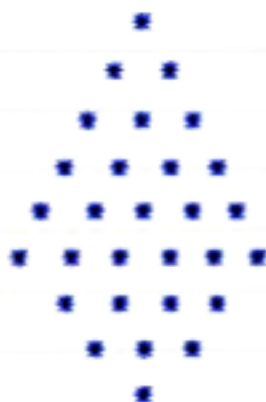
8

Diamond

```

#include <stdio.h>
int main() {
    int n = 5;
    // upper half : Regular pyramid
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    // lower half : Inverted pyramid
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}

```

O/p:-

```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5
6      // Upper half
7      for (int i = 1; i <= n; i++) {
8          for (int j = 1; j <= n - i; j++)
9              printf(" ");
10
11         for (int k = 1; k <= (2 * i - 1); k++)
12             printf("*");
13
14         printf("\n");
15     }
16
17     // Lower half
18     for (int i = n - 1; i >= 1; i--) {
19         for (int j = 1; j <= n - i; j++)
20             printf(" ");
21     }
```

```
9      |      |      printf(" ");
10
11      |      for (int k = 1; k <= (2 * i - 1); k++)
12      |      |      printf("*");
13
14      |      printf("\n");
15      |  }
16
17      |  // Lower half
18      |  for (int i = n - 1; i >= 1; i--) {
19      |      |  for (int j = 1; j <= n - i; j++)
20      |      |      |  printf(" ");
21
22      |      |  for (int k = 1; k <= (2 * i - 1); k++)
23      |      |      |  printf("*");
24
25      |      |  printf("\n");
26      |  }
27
28      |  return 0;
29      |  }
```

Output

Clear

*

*

=== Code Execution Successful ===